Huellitas — E-Commerce distribuido
Documento de seguridad (SEC-1)

Auditoría y Seguridad de Sistemas · Proyecto 2

STRIDE y DFD · OWASP Top 10 con referencias al código · PCI-DSS · BCP/DRP · ALE/ROSI · ISO 27001 · OWASP ZAP

Generado el 17 de octubre de 2026

Huellitas — Arquitectura, DFD y modelo de amenazas STRIDE

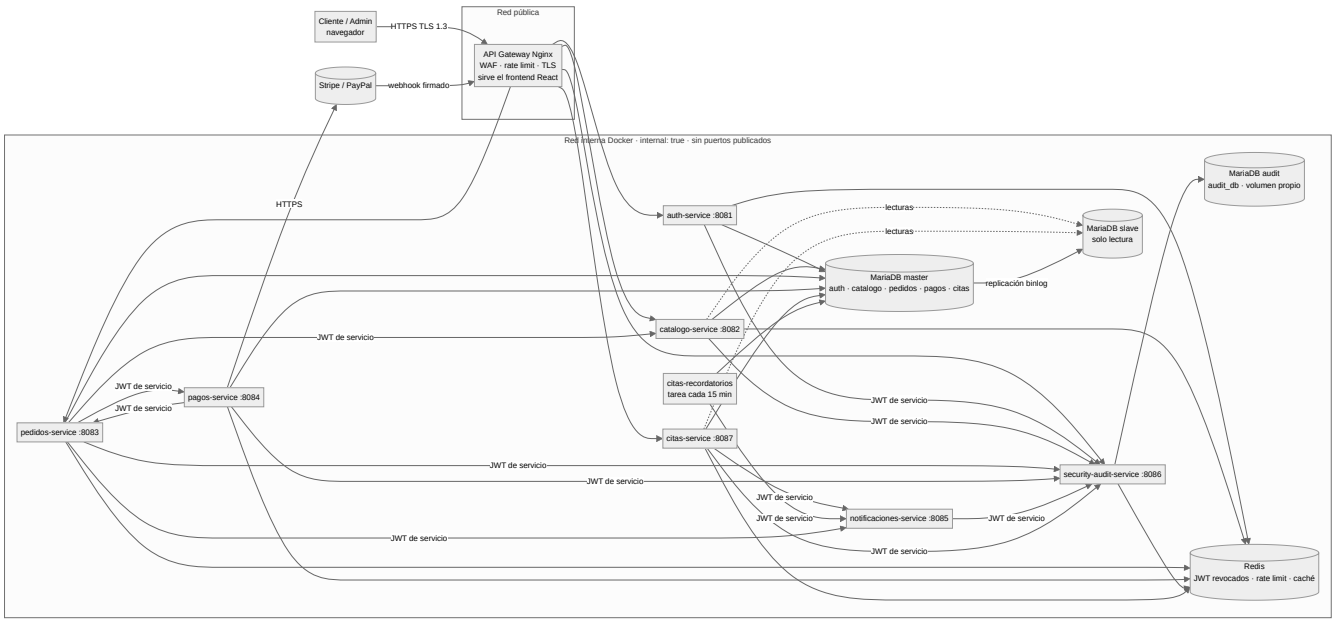
Negocio: tienda en línea de productos para mascotas (alimento, accesorios, farmacia veterinaria) de la veterinaria *Huellitas*. Es distinto del Proyecto 1 (POS de librería y papelería escolar).

Flujo de compra: catálogo → carrito → checkout → pago (Stripe / PayPal sandbox) → confirmación por email (EmailJS) → entrega (domicilio o retiro en sede).

Flujo de citas (FASE 15): servicio (la sede sale del servicio) → día → turno libre → mascota → reserva → correo con código de verificación → confirmación del admin → consulta con historial clínico y carné de vacunas → recordatorios. Las citas se pagan en la sede, así que este flujo no toca PCI-DSS.

---

1. Arquitectura de microservicios



Componente	Estado	Tecnología	Puerto
API Gateway	✅ semana 1	Nginx 1.27 Alpine	80/443 (host 8088/8443 en local)
auth-service	✅ semana 1	PHP 8.3 + Apache (www-data) · auth_db	8081 interno
security-audit-service	✅ semana 1	PHP 8.3 + Apache (www-data) · audit_db	8086 interno
MariaDB master / slave	✅ semana 1	MariaDB 10.6, replicación por binlog	3306 interno
MariaDB audit	✅ semana 1	MariaDB 10.6, volumen `audit_data`	3306 interno
Redis	✅ semana 1	Redis 7, AOF, contraseña	6379 interno
catalogo-service	✅ semana 2	PHP 8.3 · catalogo_db (lee de la réplica) · Redis · Cloudinary	8082 interno

Componente	Estado	Tecnología	Puerto
pedidos-service	✅ semana 2	PHP 8.3 · pedidos_db	8083 interno
pagos-service	✅ semana 3	PHP 8.3 · pagos_db · Stripe/PayPal · red `salida`	8084 interno
notificaciones-service	✅ semana 3	PHP 8.3 · EmailJS · OpenWeatherMap/Open-Meteo · red `salida`	8085 interno
citas-service	✅ FASE 15	PHP 8.3 · citas_db (disponibilidad desde la réplica) · Redis · sin salida a Internet	8087 interno
citas-recordatorios	✅ FASE 15	Misma imagen que citas-service; tarea periódica sin puerto	—

Con `citas-service` la arquitectura queda en **7 microservicios** (auth, catálogo, pedidos, pagos, notificaciones, auditoría y citas), más dos tareas periódicas (`pedidos-vencimientos` y `citas-recordatorios`).

Zero Trust

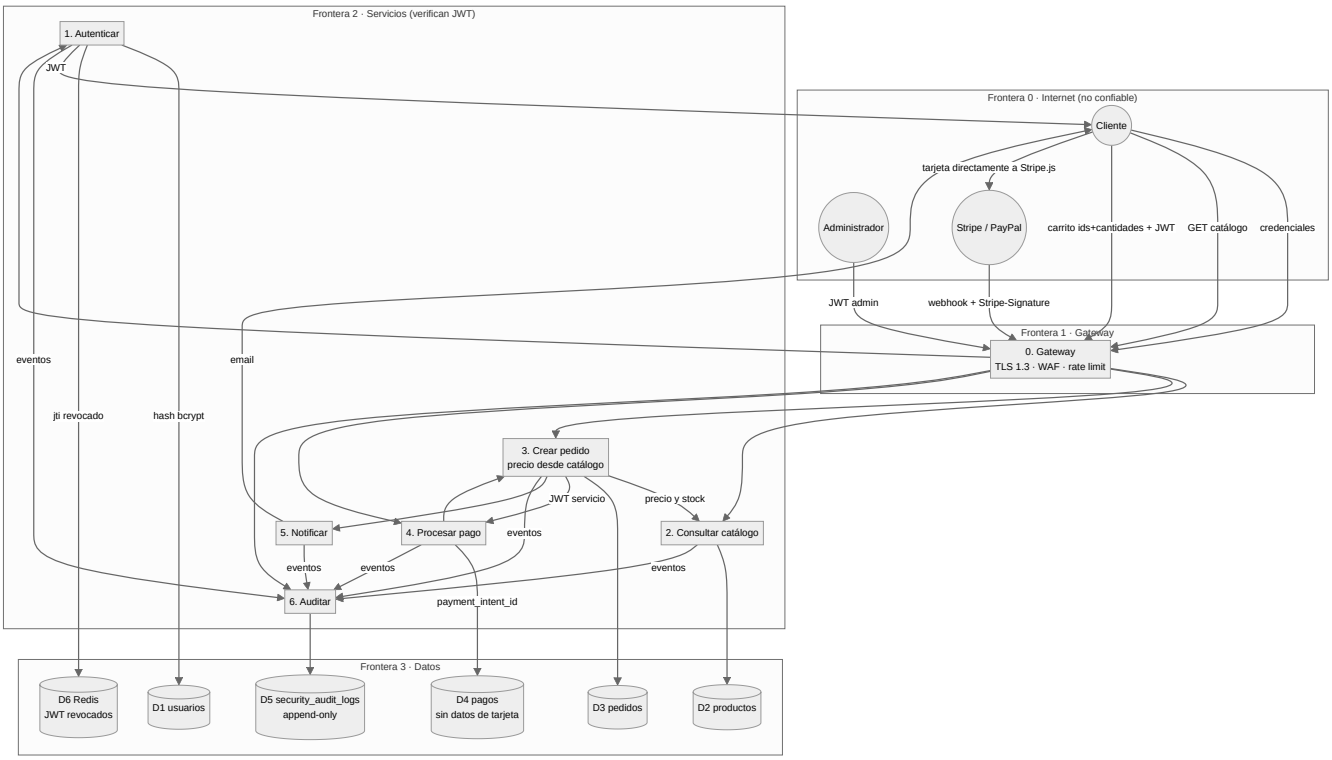
Ningún servicio confía en la red interna. Cada petición, sea del navegador o de otro servicio, trae un JWT que se verifica en el servicio que la recibe:

- **JWT de usuario** (`typ=user`, firmado con `JWT_SECRET`, 1 h). El servicio verifica la firma HS256, `exp` e `iss`, y comprueba que el `jti` no esté en la lista negra de Redis. El rol sale del token firmado. Código: `services/shared/src/Guard.php::usuario()`.
- **JWT de servicio** (`typ=service`, firmado con un secreto distinto, `SERVICE_JWT_SECRET`, 60 s, de un solo uso). `sub` identifica al servicio que llama, y el receptor tiene una lista de servicios permitidos. Código: `Guard::servicio()`.
- Si Redis no está disponible, la verificación **falla cerrada** (503): sin la lista negra, el servicio no acepta tokens.

---

2. Diagrama de flujo de datos (DFD) del ciclo de compra

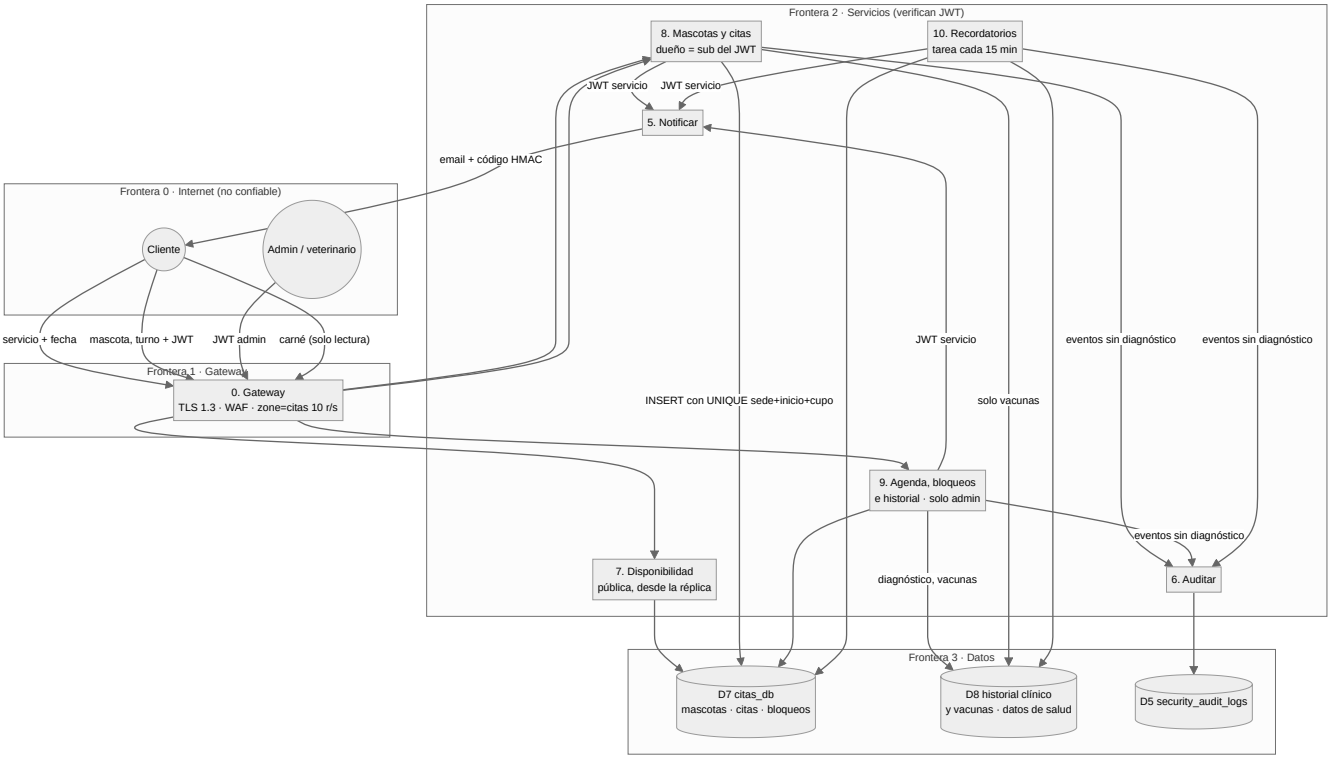
Las líneas punteadas son **fronteras de confianza**. Cada flujo que cruza una frontera tiene un control asociado.



Puntos clave del DFD

1. **El número de tarjeta nunca cruza nuestras fronteras.** Va del navegador a Stripe.js o PayPal. Nosotros guardamos solo el `payment_intent_id`.
2. **El cliente manda IDs de producto y cantidades, no precios.** `pedidos-service` consulta el precio a `catalogo-service` en el servidor.
3. Los servicios se identifican entre sí con un JWT de servicio de un solo uso, porque la red interna no se considera confiable.

2.1 DFD del flujo de citas



Puntos clave del flujo de citas

1. **El dueño sale del token.** Mascotas, citas y carné se filtran por `id_cliente = sub`. Si el recurso es de otro cliente se responde 404, igual que si no existiera, y queda `ACCESO_DENEGADO`.
2. **La base decide quién se queda el turno.** El índice `UNIQUE (sede, inicio, cupo)` impide la doble reserva aunque lleguen varias peticiones a la vez. No depende de que el código lea la disponibilidad antes.
3. **Datos de salud separados.** El diagnóstico y el tratamiento solo los ve el admin, y cada lectura se audita. El cliente ve el carné de vacunas sin diagnósticos. La bitácora guarda qué se registró (vacunas, peso), no el diagnóstico.
4. **La tarea de recordatorios no tiene puerto.** El script está fuera de `public/`, así que nadie lo puede llamar por HTTP.

---

3. Modelo STRIDE

STRIDE	Flujo / activo	Ataque	Contramedida	Dónde está	Estado
S Spoofing	Login (P1)	Credential stuffing, fuerza bruta	bcrypt cost 12; bloqueo de la cuenta 15 min tras 5 fallos; 50 intentos por IP cada 15 min; 20 req/min por IP en `/login` en el gateway; si la misma cuenta falla desde ≥3 IPs, el evento se marca `FRAUDE`; mismo mensaje y mismo tiempo de respuesta exista o no el correo	`services/auth/src/AuthController.php` (`login()`, `fallo()`, `HASH_SENUELO`), `gateway/default.conf.template` (`zone=login`)	✅
S Spoofing	Servicio → servicio	Un contenedor comprometido se hace pasar por otro	JWT de servicio con secreto aparte, 60 s, de un solo uso (anti-replay en Redis); lista de servicios permitidos	`services/shared/src/Guard.php::servicio()`	✅
T Tampering	Carrito → pedido (P3)	Cambiar el precio en `POST /api/pedidos`	Solo se reenvían `variante_id` y `cantidad`; el precio se lee del maestro en `catalogo-service`; descuento y envío se calculan en el servidor, en centavos	`PedidosController::cotizarEnCatalogo()`, `StockController::cotizar()`, `Totales.php`	✅

STRIDE	Flujo / activo	Ataque	Contramedida	Dónde está	Estado
T Tampering	Token (P1)	Editar el payload del JWT o usar `alg=none`	Solo HS256; firma comparada en tiempo constante (`hash_equals`)	`services/shared/src/Jwt.php::verificar()`	✅
T Tampering	Webhook de pago (P4)	Enviar un `payment_intent.succeeded` falso o reenviar uno viejo	Firma `Stripe-Signature` (HMAC, tolerancia 5 min), idempotencia por `event_id` , verificación de monto, moneda y pedido	`Stripe::firmaValida()` , `PagosController::webhookStripe()`	✅
T Tampering	Bitácora (D5)	Borrar o alterar los rastros	Triggers que prohíben UPDATE/DELETE incluso a root; `audit_user` solo con INSERT/SELECT; cadena de hashes SHA-256 verificable	`db/audit/01-audit_db.sh` , `AuditController::integridad()`	✅
R Repudiation	Pedido (P3)	El cliente niega haber hecho un pedido	`PEDIDO_CREADO` con cliente, IP, user-agent, líneas y precios; historial de estados con quién los cambió	`PedidosController::crear()` , tabla `historial_estados`	✅
R Repudiation	Pago (P4)	El cliente niega haber pagado	`PAGO_INICIADO` y `PAGO_CONFIRMADO` con el `payment_intent_id` de Stripe (inmutable del lado de Stripe), IP y user-agent; email con código de verificación HMAC	`PagosController` , `NotificacionesController::codigo()`	✅
R Repudiation	Cualquier acción	Un servicio reporta eventos que no le corresponden	`servicio` se toma del token, no del cuerpo; cada código de evento tiene sus servicios permitidos; la hora la pone el audit-service	`AuditController::EVENTOS`	✅
I Info Disclosure	Pedidos (D3)	`GET /api/pedidos` de otros clientes, IDs secuenciales	`WHERE id_cliente = <sub del JWT>` en todas las consultas; pedidos con **UUID v4**; 404 también cuando es ajeno, y queda registrado `ACCESO_DENEGADO`	`PedidosController::misPedidos()` , `ver()`	✅
I Info Disclosure	Errores	Stack traces, versión del servidor	`display_errors=Off` ; errores genéricos en JSON; `server_tokens off` ; `ServerTokens Prod` ; sin `X-Powered-By`	`services/shared/src/Router.php` , `services/Dockerfile`	✅
I Info Disclosure	Tránsito	Sniffing	Solo TLS 1.3, HSTS, HTTP → 301 a HTTPS	`gateway/default.conf.template`	✅
D DoS	Catálogo (P2)	Scraping masivo o inundación	Rate limit por IP y por servicio, `limit_conn` , catálogo servido desde Redis y leído de la réplica, CDN de Cloudinary, `mem_limit` / `cpu_shares` por contenedor; el stock exacto no se publica	`gateway/default.conf.template` , `CatalogoController::listar()`	✅
S Spoofing	Correo de la cita	Correo falso que dice venir de Huellitas, o un cliente que niega haber reservado	Código HMAC `cita|id|inicio` con `COMPROBANTE_SECRET` en el correo y en «Mis citas»; solo `citas-service` puede pedir correos de citas a notificaciones	`CitasController::notificar()` , `NotificacionesController`	✅ CP-42
T Tampering	Reserva (P8)	Reservar dos veces el mismo cupo enviando peticiones a la vez	`UNIQUE (sede, inicio, cupo)` y `UNIQUE (mascota_id, inicio)` en la BD; se prueba cada cupo dentro de una transacción y el índice decide	`CitasController::crear()` , `db/master/07-citas_db.sql`	✅ CP-39
T Tampering	Bloqueo de turnos (P9)	Una reserva que entra mientras el admin bloquea el turno	`bloquear()` toma el rango con `FOR UPDATE` y la reserva vuelve a leer los bloqueos antes de confirmar; no se bloquea un rango con citas activas	`CitasController::bloquear()` , `crear()`	✅ CP-43
R Repudiation	Citas y consultas	El cliente niega haber reservado o cancelado; el admin niega un cambio de estado o un diagnóstico	`CITA_CREADA` , `CITA_CANCELADA` y `CITA_ESTADO_CAMBIADO` con estado anterior y nuevo; tabla `historial_citas` con quién hizo cada cambio; `HISTORIAL_REGISTRADO` con autor, vacunas y peso	`CitasController::moverEstado()` , `registrarHistorial()`	✅ CP-40, CP-44

STRIDE	Flujo / activo	Ataque	Contramedida	Dónde está	Estado
I Info Disclosure	Mascotas, citas y carné (D7)	Ver las mascotas, el teléfono o las vacunas de otro cliente cambiando el ID	Filtro por `sub` del JWT, IDs UUID v4, 404 cuando es ajeno y `ACCESO_DENEGADO` en la bitácora	`mascotaPropia()`, `citaPropia()`, `carne()`	✅ CP-37, CP-45
I Info Disclosure	Historial clínico (D8)	Un cliente lee diagnósticos, o la bitácora se vuelve una copia de los datos de salud	Historial solo con `Guard::usuario(['admin'])` y cada lectura auditada (`HISTORIAL_CONSULTADO`); el carné no trae diagnóstico ni tratamiento; la bitácora no los copia (minimización, A.5.34)	`historialAdmin()`, `carne()`, `registrarHistorial()`	✅ CP-44, CP-45
D DoS	Agenda (P8)	Un cliente reserva todos los turnos para bloquear la agenda	Máximo 5 citas próximas por cliente; `zone=citas` (10 r/s, ráfaga 40) en el gateway; reservas hasta 60 días adelante y con 1 h de anticipación	`CitasController::MAX_ACTIVAS`, `gateway/default.conf.template`	✅ CP-41
E Elevation of Privilege	Agenda, bloqueos e historial	Un cliente cambia estados, bloquea turnos o registra consultas	Todas las rutas `/api/citas/admin/*` exigen `Guard::usuario(['admin'])`; las transiciones de estado salen de una tabla fija	`CitasController::TRANSICIONES`, `cambiarEstado()`	✅ CP-40, CP-43
E Elevation of Privilege	Panel admin	Manipular el JWT o enviar `rol=admin` al registrarse	El rol solo sale del token firmado; el registro público siempre crea `cliente`; `Guard::usuario(['admin'])` en cada ruta administrativa; intento de cliente en el panel → `ACCESO_ADMIN_PANEL FALLO`	`AuthController::registro()`, `accesoAdmin()`, `Guard.php`	✅

Riesgo residual que se acepta

- El WAF de Nginx usa firmas (lista negra) y puede saltarse con codificaciones raras. Por eso hay una segunda capa en PHP (`Validar::sinAtaques`) y, sobre todo, el 100 % de las consultas usa prepared statements reales (`PDO::ATTR_EMULATE_PREPARES => false`).
- El JWT revocado se busca en Redis. Si Redis cae, los servicios devuelven 503 en lugar de aceptar tokens sin verificar: se prefiere la disponibilidad reducida a la seguridad reducida.

---

4. Evidencia automática

`tests/seguridad.sh` verifica contra el stack levantado los controles marcados con ✅: TLS, cabeceras, JWT, bloqueo, WAF, Zero Trust, bitácora append-only, hardening Docker, replicación y el módulo de citas (CP-37 a CP-46).

```
docker compose up -d --build
tests/seguridad.sh
```

Huellitas — Controles OWASP Top 10 (2021)

Cada control indica **dónde está en el código** y **qué caso de `tests/seguridad.sh` lo comprueba**. Estado al cierre de la semana 3: los 7 controles (A01, A02, A03, A05, A07, A08 y A09) implementados. En la FASE 15 se aplicaron los mismos controles al módulo de citas (`citas-service`), con las pruebas CP-37 a CP-46.

ID	Amenaza	Control	Código	Prueba
A01	Broken Access Control	`id_cliente` del JWT en cada consulta de pedidos, mascotas, citas y carné; 404 idéntico para "no existe" y "es de otro"; rutas admin (incluida la agenda y el historial clínico) con `Guard::usuario(['admin'])`	`services/pedidos/src/PedidosController.php` (`misPedidos`, `ver`), `services/citas/src/CitasController.php` (`mascotaPropia`, `citaPropia`), `services/shared/src/Guard.php`	CP-07, CP-20, CP-37, CP-40, CP-45
A02	Cryptographic Failures	bcrypt cost 12; solo TLS 1.3 + HSTS; sin datos de tarjeta en ninguna BD; JWT HS256 con secretos de 48 caracteres en `.env`	`services/auth/src/AuthController.php` (`BCRYPT`), `gateway/default.conf.template` (`ssl_protocols`), `db/master/*.sql`	CP-01, CP-02, CP-05
A03	Injection	PDO sin emulación de prepared statements; validación de tipo y rango; WAF en Nginx (URL, cabeceras) y en PHP (cuerpo JSON, también en mascotas, citas y diagnósticos)	`services/shared/src/Db.php`, `services/shared/src/Validar.php`, `gateway/default.conf.template` (`map $waf_*`)	CP-10, CP-24, CP-37, CP-44
A05	Security Misconfiguration	Secretos solo en `.env`; BD y Redis sin puertos publicados; contenedores PHP como `www-data`, `read_only`, `cap_drop: ALL`; sin versiones en cabeceras; errores genéricos	`docker-compose.yml`, `services/Dockerfile`, `services/shared/src/Router.php`	CP-03, CP-14
A07	Identification & Authentication Failures	5 intentos → bloqueo 15 min; rate limit en `/login`; JWT de 1 h; logout → lista negra Redis; mismo mensaje y mismo tiempo exista o no el correo; **verificación en dos pasos TOTP** (secreto cifrado AES-256-GCM, desafío de 5 min y un solo uso, códigos no reutilizables, los códigos fallidos cuentan para el bloqueo)	`AuthController::login()`, `verificarMfa()`, `Totp.php`, `Cache::revocar()`, `zone=login`	CP-05, CP-08, CP-09, CP-16, CP-34
A08	Software & Data Integrity Failures	Firma `Stripe-Signature` del webhook (HMAC, tolerancia 5 min, idempotencia) y verificación de monto; cadena de hashes en la bitácora; dependencias fijadas con `package-lock.json`	`services/pagos/src/Stripe.php::firmaValida()`, `AuditController::integridad()`	CP-12, CP-26, CP-27
A09	Security Logging & Monitoring Failures	`security-audit-service` con eventos append-only; cada servicio reporta los suyos (27 tipos: los 14 obligatorios y 13 extra, 10 de ellos del módulo de citas)	`services/audit/src/AuditController.php`, `services/shared/src/Auditoria.php`	CP-12, CP-13, CP-40, CP-46

A01 — Broken Access Control

Riesgo en el e-commerce: un cliente cambia el ID de la URL (`/api/pedidos/…`) y ve la dirección, el teléfono y la compra de otra persona. Otro riesgo es un cliente que llama a rutas de administración.

Controles:

1. **El filtro sale del token, nunca de un parámetro.** `misPedidos()` hace `WHERE p.id_cliente = ?` con `$claims['sub']`, el ID que viene dentro del JWT firmado. El cliente no tiene forma de pedir los pedidos de otro.

2. **Detalle de un pedido.** `ver()` busca primero el dueño. Si no coincide con el `sub` del token y el rol no es admin, responde **404**, igual que si el pedido no existiera, para no confirmar que existe. Además registra `ACCESO_DENEGADO` en la bitácora, lo que permite detectar a alguien probando IDs.
3. **IDs no adivinables.** Los pedidos usan UUID v4 (`Jwt::uuid()`, 122 bits aleatorios), no enteros consecutivos.
4. **Rutas administrativas.** Precios, stock, estados de pedido y bitácora exigen `Guard::usuario(['admin'])`. El rol viene del token firmado, y el registro público siempre crea `cliente` aunque el cuerpo diga `"rol":"admin"`.
5. **Servicio a servicio.** `/internal/*` no se publica en el gateway, y cada ruta interna acepta solo a ciertos servicios. Por ejemplo, solo `pedidos-service` puede reservar stock.

```
// services/pedidos/src/PedidosController.php — misPedidos()
$consulta = Db::lectura()->prepare(self::SQL_RESUMEN . ' WHERE p.id_cliente = ? GROUP BY p.id ORDER BY p.fecha
DESC LIMIT 100');
$consulta->execute([(int) $claims['sub']]);
```

Evidencia: CP-20. El cliente B pide el pedido del cliente A y recibe 404, y la bitácora registra `ACCESO_DENEGADO` con el ID del pedido y el correo de B.

A01 en el módulo de citas

Las mascotas, las citas y el carné de vacunas siguen la misma regla que los pedidos:

- **Dueño desde el token.** `misCitas()` y `mascotas()` filtran por `id_cliente = sub`. `mascotaPropia()` y `citaPropia()` responden 404 si el recurso es de otro cliente y registran `ACCESO_DENEGADO` con el recurso (`mascota`, `cita` o `carné de vacunas`).
- **No se reserva con la mascota de otro.** `crear()` valida la mascota con `mascotaPropia()` antes de tocar la agenda.
- **Rutas del veterinario.** La agenda, el cambio de estado, los bloqueos de turnos y el historial clínico están bajo `/api/citas/admin/*` y exigen `Guard::usuario(['admin'])`. El historial clínico es dato de salud: cada lectura queda como `HISTORIAL_CONSULTADO`.
- **Solo transiciones válidas.** El estado de una cita solo avanza según la tabla `TRANSICIONES` (`pendiente → confirmada → atendida`, `no_asistio` o `cancelada`). Un estado final no se puede cambiar.

Evidencia: CP-37 (editar una mascota ajena → 404 + `ACCESO_DENEGADO`), CP-38 (reservar con la mascota de otro → 404), CP-40 (un cliente no ve la agenda ni cambia estados → 403) y CP-45 (carné ajeno → 404; historial clínico desde una cuenta de cliente → 403).

A02 — Cryptographic Failures

Dato	Protección
Contraseñas	`password_hash(..., PASSWORD_BCRYPT, ['cost' => 12])`. Si en el futuro sube el costo, `password_needs_rehash` actualiza el hash en el siguiente login
Tránsito	Gateway con `ssl_protocols TLSv1.3` únicamente, HSTS de 1 año y HTTP → 301 a HTTPS
Tarjetas	**Ninguna tabla tiene columnas de tarjeta.** El navegador envía la tarjeta directo a Stripe.js / PayPal (semana 3) y Huellitas guarda solo `payment_intent_id`
Tokens	JWT HS256 con `JWT_SECRET` para usuarios y otro secreto distinto (`SERVICE_JWT_SECRET`) entre servicios. La firma se compara con `hash_equals`
Bitácora	Cadena SHA-256: cada fila incluye el hash de la anterior
Secretos	Solo en `.env` (permisos 600, en `.gitignore`). `.env.example` trae las claves vacías

---

A03 — Injection

Capa 1: consultas parametrizadas. Todas las consultas pasan por `PDO::prepare` con `PDO::ATTR_EMULATE_PREPARES => false`, así que el motor recibe la consulta y los datos por separado. Hay dos casos donde se arma texto SQL, y en ninguno entra texto del usuario:

- Los nombres de columna en `UPDATE … SET` salen de una lista fija del código (`actualizarVariante`, `actualizarProducto`).
- Los `IN (?, ?, …)` se arman solo con el número de signos `?`.

Capa 2: validación de tipo y rango.

- `variante_id` y `cantidad` deben ser enteros JSON. La cantidad va de 1 a 99 y el carrito tiene como máximo 30 líneas.
- Los precios son números positivos con hasta 2 decimales.
- Teléfono, cupón y UUID se validan con expresiones regulares.
- Los textos se recortan, se les quitan las etiquetas y se limita su longitud.

Capa 3: WAF.

- Nginx bloquea con 403 las firmas conocidas en la URL, el user-agent, el referer y las cookies: `UNION SELECT`, `' OR 1=1`, `<script`, `../`, `sleep(`, `sqlmap`…
- Nginx no inspecciona el cuerpo, así que `Validar::sinAtaques()` revisa todos los textos del JSON salvo las contraseñas.

Evidencia: CP-10 y CP-24. `x'; DROP TABLE pedidos; --` en la dirección da 400, `<img onerror>` en las notas da 400 y `"1 OR 1=1"` como ID de variante también se rechaza.

Manipulación de precios (T de STRIDE, relacionado con A04 Insecure Design)

El navegador manda **solo** `variante_id` y `cantidad`. `pedidos-service` descarta cualquier otro campo (`precio`, `subtotal`, `total`) antes de llamar a `catalogo-service`. El catálogo devuelve el precio vigente desde el **maestro** (no la réplica, para no usar un precio desactualizado) y `Totales::calcular()` hace las cuentas en centavos enteros. El cupón se valida en la BD y el envío se decide en el servidor.

Evidencia: CP-18. Se envía `"precio": 0.01, "total": 0.02` y el pedido queda con el precio real del catálogo.

Doble reserva y acaparamiento de la agenda (A04 Insecure Design)

Riesgo: dos clientes reservan el último cupo al mismo tiempo y los dos reciben la cita, o un cliente reserva todos los turnos de una sede.

Controles (`CitasController::crear()`, `db/master/07-citas_db.sql`):

1. **El índice único decide, no el código.** La tabla `citas` tiene `UNIQUE (sede, inicio, cupo)`. La reserva prueba el cupo 1 y luego el 2 dentro de una transacción; si los dos ya están ocupados, el motor rechaza el INSERT y el cliente recibe 409. Leer la disponibilidad antes de reservar no basta, porque dos peticiones pueden leer «1 libre» a la vez.
2. **Una mascota, una cita por hora:** `UNIQUE (mascota_id, inicio)`.
3. **Bloqueos sin carreras.** `bloquear()` toma el rango con `SELECT … FOR UPDATE`. Una reserva que llega mientras tanto espera, vuelve a leer los bloqueos y se deshace si el turno quedó bloqueado.
4. **Límite por cliente:** máximo 5 citas próximas, reservas hasta 60 días adelante y al menos 1 h antes del turno. En el gateway, `zone=citas` permite 10 peticiones por segundo por IP (ráfaga de 40).

Evidencia: CP-39 manda 3 reservas simultáneas para 2 cupos: entran 2, la tercera recibe 409 y en la BD hay exactamente 2 citas. CP-41 comprueba que la 6.ª cita próxima de un cliente se rechaza. CP-43 comprueba que no se reserva en un turno bloqueado.

A08 — Software & Data Integrity Failures

Riesgo: alguien llama al webhook con un `payment_intent.succeeded` inventado para que un pedido figure como pagado sin pagar, o reenvía un webhook real capturado antes.

Controles (`services/pagos/src/Stripe.php`, `PagosController::webhookStripe()`):

1. Sin `Stripe-Signature` válida no se procesa nada. Se calcula `HMAC-SHA256(t + "." + cuerpo_crudo, STRIPE_WEBHOOK_SECRET)` y se compara con `hash_equals` contra cada `v1`.
2. Si `t` tiene más de 300 s de diferencia con la hora del servidor, se rechaza (anti-replay).
3. `webhook_eventos.event_id` es la clave primaria: un evento repetido responde 200 sin reprocesarse.
4. Aunque la firma sea válida, el pago solo se confirma si `amount_received`, `currency` y `metadata.pedido_id` coinciden con lo guardado. Si no coinciden, queda `PAGO_FALLIDO` con resultado `FRAUDE`.
5. Todo intento rechazado queda en la bitácora: `WEBHOOK_STRIPE_RECIBIDO` con FALLO (sin firma) o FRAUDE (firma falsa).

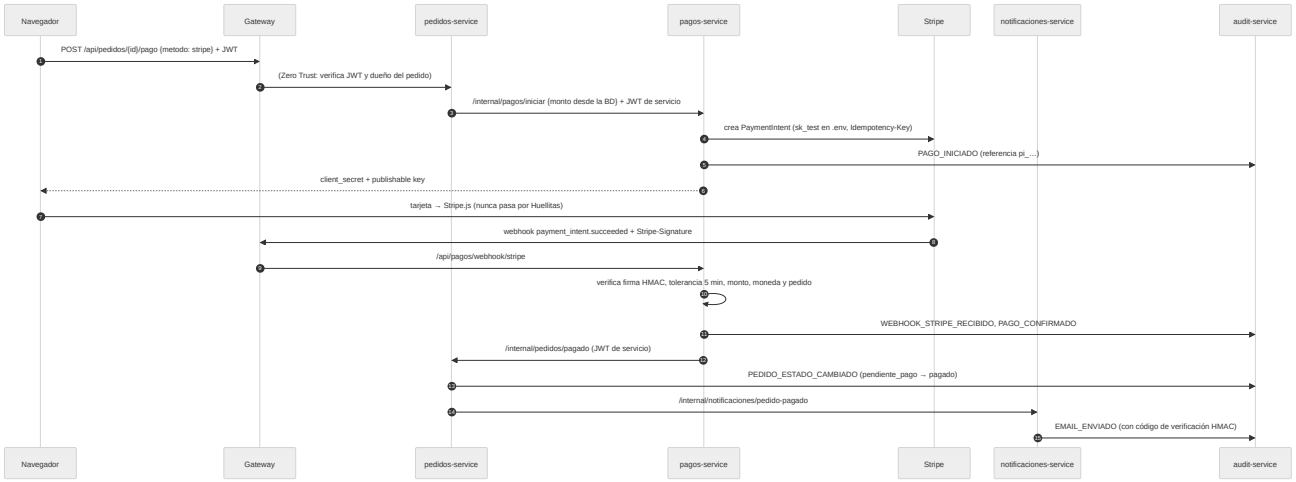
Evidencia: CP-26 cubre sin firma, firma falsa, firma vieja, cuerpo alterado, firma válida y evento repetido. CP-27 cubre el monto manipulado y la tarjeta rechazada.

Ver también `03-pci-dss-pagos.md`.

Huellitas — Seguridad del módulo de pagos (PCI-DSS básico)

Huellitas cobra con **Stripe** (tarjeta) y **PayPal**, ambos en modo sandbox. El diseño busca que el sistema **nunca toque los datos de tarjeta**: así queda en el alcance más reducido de PCI-DSS (SAQ A) y una brecha de nuestras bases de datos no expondría tarjetas.

Flujo



PayPal sigue el mismo esquema: `pagos-service` crea la orden (Orders v2) y la **captura del lado del servidor**. Solo la da por buena si `status = COMPLETED`, el monto y la moneda coinciden con el pago guardado, y el `custom_id` es el pedido correcto.

Los 6 requisitos

#	Requisito	Implementación	Código	Prueba
1	**No almacenar datos de tarjeta**	`pagos` guarda solo `payment_intent_id` (o el Order ID de PayPal), el estado de la pasarela y el monto. Ninguna tabla de las 4 bases tiene columnas de número, CVV o vencimiento	`db/master/05-pagos_db.sql`	CP-28
2	**Cifrado en tránsito**	Gateway solo con TLS 1.3, HSTS y HTTP → 301. Las llamadas a Stripe/PayPal/EmailJS fuerzan HTTPS (`CURLPROTO_HTTPS`) y verifican el certificado	`gateway/default.conf.template` , `services/pagos/src/Pasarela.php`	CP-01, CP-02
3	**Control de acceso**	`/internal/pagos/*` solo acepta a `pedidos-service` con JWT de servicio de un solo uso. El gateway no publica `/api/pagos/*` , salvo el webhook. El cliente solo puede pagar **sus** pedidos, y el monto sale de la BD, no del navegador	`PagosController::iniciar()` , `PedidosController::iniciarPago()` , `propio()`	CP-29
4	**Logging de transacciones**	`PAGO_INICIADO` (éxito y fallo), `PAGO_CONFIRMADO` , `PAGO_FALLIDO` (FALLO o FRAUDE) y `WEBHOOK_STRIPE_RECIBIDO` , todos con la referencia de la pasarela, el `pago_id` y el `pedido_id`	`PagosController`	CP-26, CP-27
5	**Verificación de webhooks**	`Stripe-Signature` : HMAC-SHA256 de `t.payload` con el Webhook Secret, comparación en tiempo constante y tolerancia de 300 s (anti-replay). Sin firma → 400 + evento FALLO; firma falsa → 400 + evento FRAUDE. Cada evento se procesa una sola vez (`webhook_eventos`)	`services/pagos/src/Stripe.php::firmaValida()`	CP-26

#	Requisito	Implementación	Código	Prueba
6	**API keys en entorno**	`STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `PAYPAL_CLIENT_SECRET` y `EMAILJS_PRIVATE_KEY` solo en `.env` (en `.gitignore`), inyectadas únicamente en el contenedor que las usa. El navegador recibe solo las claves públicas (`pk_test`, `client-id`)	`docker-compose.yml`, `.env.example`	CP-30

Controles adicionales

- **Monto manipulado en el webhook.** Aunque la firma sea válida, si `amount_received`, la moneda o `metadata.pedido_id` no coinciden con el pago guardado, el pedido **no** se confirma y queda `PAGO_FALLIDO` con resultado `FRAUDE` (CP-27).
- **Doble cobro.** Un pedido pagado no se puede volver a pagar (409). La Idempotency-Key de Stripe y el `PayPal-Request-Id` hacen que repetir la llamada no cree otro cobro.
- **Pagado después de cancelar.** Si llega la confirmación de un pedido ya cancelado, no se reactiva. Queda en el log como `REEMBOLSO_PENDIENTE` para reembolsarlo a mano.
- **Red.** Solo `pagos-service` y `notificaciones-service` están en la red `salida`. El resto de los servicios no puede abrir conexiones a Internet (CP-30).
- **Repudio.** El correo de confirmación lleva un código `HMAC(pedido|total|referencia)`. Ante un reclamo, el admin lo recalcula con `COMPROBANTE_SECRET` (`/internal/notificaciones/verificar`).
- **Base de datos.** `pagos_user` no tiene permiso `DELETE` sobre `pagos_db`.

Evidencia para la presentación

```
-- SEC-3: la tabla de pagos no tiene datos de tarjeta
SELECT id, id_pedido, monto, metodo, payment_intent_id, stripe_status, estado, fecha_pago FROM pagos_db.pagos;
```

```
# El webhook rechaza lo que no viene firmado por Stripe
curl -k -X POST https://localhost:8443/api/pagos/webhook/stripe -d '{"type":"payment_intent.succeeded"}'
# → 400 {"error":"Firma inválida"} y un WEBHOOK_STRIPE_RECIBIDO con resultado FALLO en la bitácora

# Las claves solo están en .env
git grep -nE 'sk_test_|whsec_' -- . ':!tests'   # sin resultados
```

Probar con Stripe de verdad (sandbox)

1. En https://dashboard.stripe.com/test/apikeys, copiar `pk_test_…` y `sk_test_…` a `.env`.
2. `stripe listen --forward-to https://localhost:8443/api/pagos/webhook/stripe --skip-verify` y copiar el `whsec_…` que muestra a `STRIPE_WEBHOOK_SECRET`.
3. `docker compose up -d pagos-service` y pagar con la tarjeta de prueba `4242 4242 4242 4242` (cualquier fecha futura y cualquier CVC).

Huellitas — Plan de continuidad (BCP) y recuperación ante desastres (DRP)

Objetivo: que la tienda vuelva a vender y la clínica a recibir citas en **menos de 60 minutos** ante cualquiera de los escenarios de este plan, sin perder pagos, citas ni la trazabilidad de la bitácora.

Responsables: Administrador del sistema (titular) · Desarrollador de guardia (suplente) · Dueño del negocio (comunicación con clientes y decisión de activar el sitio B).

---

1. Análisis de impacto (BIA)

Servicio	Criticidad	RTO	RPO	Depende de	Por qué
pagos-service	CRÍTICA	10 min	0	maestro, Redis, Stripe/PayPal	Sin él no hay ventas. Los pagos viven también en Stripe/PayPal, así que se reconcilian desde la pasarela (RPO 0)
auth-service	CRÍTICA	10 min	0	maestro, Redis	Sin login no se puede comprar ni administrar
catalogo-service	CRÍTICA	15 min	5 min	réplica (o maestro), Redis	Es la vitrina. Mientras tanto la caché de Redis sigue sirviendo el catálogo
BD maestra	CRÍTICA	10 min	5 min	volumen `master_data`	Guarda usuarios, catálogo, pedidos, pagos, citas e historial clínico
pedidos-service	ALTA	20 min	5 min	maestro, catálogo, pagos	Los pedidos se pueden rehacer con la información de Stripe y del correo
citas-service	ALTA	30 min	5 min	maestro, réplica, Redis, auditoría	Sin él no se reservan citas en línea y el veterinario no ve la agenda. Mientras tanto se atiende por teléfono; las citas del día ya llegaron por correo con su código. El historial clínico no se puede rehacer de otra fuente, por eso el RPO es el de los binlogs (5 min)
citas-recordatorios	MEDIA	60 min	—	citas-service, notificaciones	Al volver se ponen al día solos: cada aviso se marca al enviarse (`recordatorio_at`, `aviso_at`) y no se repite
Redis	ALTA	15 min	0 (se recalcula)	volumen `redis_data` (AOF)	La lista negra de JWT persiste con AOF. La caché se reconstruye sola
security-audit-service	ALTA	30 min	15 min	BD de auditoría	Mientras no responde, los eventos quedan en el log del contenedor (`AUDITORIA_PENDIENTE`) y se reingresan
notificaciones-service	MEDIA	60 min	30 min	EmailJS	El correo se puede reenviar. El pedido no depende de él
Gateway	CRÍTICA	10 min	—	certificados	Sin estado: se vuelve a levantar desde la imagen

---

2. Escenarios de incidente y respuesta

Escenario 1 — Ransomware en la BD maestra · Prob. media · Impacto crítico

Fase	Acción
Detección	Healthcheck del maestro en rojo, 500 en `/api/pedidos/*`, archivos `.ibd` ilegibles
Contención	Aislar el host (sacarlo de la red), **no pagar**, preservar el volumen para el análisis forense (`docker volume` → snapshot)
Recuperación	Maestro nuevo con volumen limpio → `ops/restore.sh <último respaldo> negocio`, que restaura el respaldo completo **y reaplica los binlogs archivados** (PITR, RPO ≤ 5 min; con `--hasta` se puede parar justo antes del ataque) → réplica nueva (se pone al día sola) → invalidar la caché del catálogo. Si el host entero está comprometido, todo se levanta en el **sitio B**

Fase	Acción
Verificación	Conteos iguales a los del último respaldo, cadena de hashes íntegra, prueba de compra
Probado	✅ `ops/simulacro-drp.sh`: respaldo completo → actividad posterior → archivado de binlogs → destrucción del maestro y la réplica → restauración con PITR. El cambio posterior al respaldo se recupera, el RTO medido es de segundos y no se pierden datos (informes)

Escenario 2 — DDoS al API Gateway · Prob. alta · Impacto alto

- **Prevención:** Cloudflare (plan gratuito) delante del dominio, en modo proxy, con "Under Attack" disponible. En Nginx: rate limit por IP y por servicio, `limit_conn 50`, `client_body_timeout 10s` y `client_max_body_size 1m`. Además, el catálogo se sirve desde Redis y la réplica, y cada contenedor tiene `mem_limit` / `cpu_shares` para que uno saturado no tumbe a los demás.
- **Respuesta:** activar "Under Attack" o un challenge para `/api/*`, bajar los límites de `zone=catalogo` y `zone=login`, bloquear ASN o países en Cloudflare, y escalar horizontalmente las réplicas del catálogo (`docker compose up --scale`).
- **RTO objetivo:** 15 min.

Escenario 3 — Fuga de datos de clientes · Prob. baja · Impacto crítico

- **Alcance limitado por diseño:** no hay tarjetas (PCI), las contraseñas son bcrypt de costo 12 y la exportación es de datos mínimos. Los datos de salud de las mascotas (diagnóstico y tratamiento) solo están en `citas_db`, solo los lee el admin y la bitácora no los copia.
- **Respuesta:**
  1. Aislar el servicio comprometido (se le quita la red `interna`).
  2. Rotar `JWT_SECRET`. Eso invalida **todas** las sesiones.
  3. Rotar las contraseñas de BD y las claves de API.
  4. Revisar la bitácora: `EXPORTACION_CLIENTES`, `ACCESO_DENEGADO`, `ACCESO_ADMIN_PANEL`, `HISTORIAL_CONSULTADO` y `LOGIN_CLIENTE_FALLIDO` con resultado `FRAUDE`.
  5. Verificar `/api/auditoria/integridad`.
- **Notificación:** a los afectados y a la autoridad en **menos de 72 h** (buena práctica, alineada con el GDPR), con qué datos salieron y qué hacer (cambiar la contraseña).

Escenario 4 — Caída del hosting principal · Prob. baja · Impacto alto

- **Sitio B:** un segundo proveedor (por ejemplo Railway o un VPS de otro proveedor) con el mismo `docker-compose.yml` + `docker-compose.prod.yml`, el `.env` guardado en el gestor de secretos y el último respaldo de R2/S3.
- **Conmutación:**
  1. `docker compose up -d` en el sitio B.
  2. `ops/restore.sh` con el último respaldo remoto.
  3. Cambiar el registro DNS en Cloudflare. Con TTL de 60 s y proxy activo, el cambio es inmediato.
- **RTO objetivo:** 45 min. **RPO:** el del último respaldo remoto (≤ 4 h; 5 min con binlogs remotos).

Escenario 5 — Certificado TLS vencido · Prob. alta · Impacto alto

- **Prevención:** Let's Encrypt con renovación automática (certbot cada 12 h, renueva a los 60 días de los 90) y recarga de Nginx después de renovar. Alerta 14 días antes del vencimiento (correo de Let's Encrypt + monitor externo).
- **Respuesta:** `certbot renew --force-renewal` + `docker compose exec gateway nginx -s reload`. Si falla el desafío HTTP-01, usar DNS-01 con el token de Cloudflare.
- **RTO objetivo:** 15 min.

Escenario 6 — Credenciales de administrador comprometidas · Prob. media · Impacto crítico

- **Detección:** `LOGIN_CLIENTE_FALLIDO` marcado `FRAUDE` (la misma cuenta desde 3 o más IPs), `ACCESO_ADMIN_PANEL` desde una IP o un navegador desconocido, `PRECIO_MODIFICADO`, `EXPORTACION_CLIENTES`, `TURNO_BLOQUEADO` o muchas

`HISTORIAL_CONSULTADO` seguidas fuera de horario.

- **Respuesta:**
  1. Revocar los JWT activos. El JWT de 1 h más la lista negra en Redis cortan el acceso; si hace falta cortarlo ya mismo, se rota `JWT_SECRET`.
  2. Desactivar la cuenta (`usuarios.activo = 0`) y forzar el cambio de contraseña.
  3. Revisar en la bitácora todo lo que hizo la cuenta desde el primer acceso sospechoso: precios con `dato_anterior`, estados de pedidos y de citas, bloqueos de turnos, historiales consultados y exportaciones.
  4. Revertir los cambios.
- **RTO objetivo:** 30 min.

---

3. Estrategia de respaldo

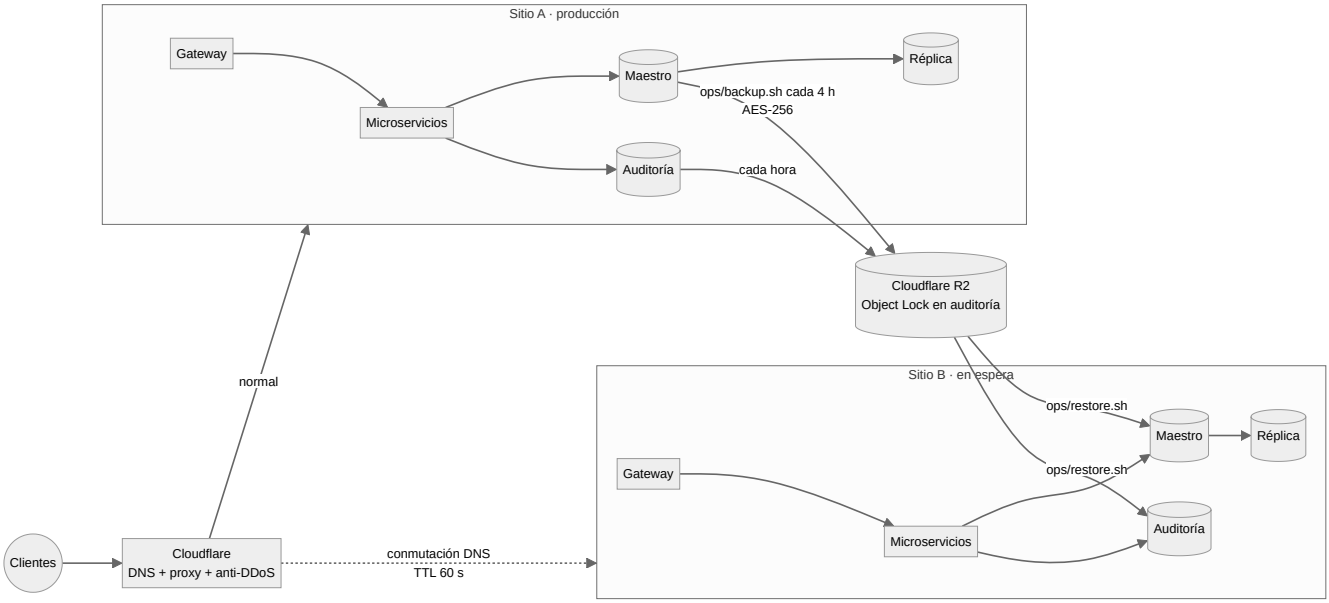
Componente	Tipo	Frecuencia	Retención	Dónde	Cómo
BD de negocio (maestro): `auth_db`, `catalogo_db`, `pedidos_db`, `pagos_db` y `citas_db`	Completo `mariadb-dump --single-transaction --master-data=2`	Diario 03:00 + cada 4 h	30 días	Local cifrado + Cloudflare R2	`ops/backup.sh` (cron), AES-256-CBC + PBKDF2, SHA-256
Binlogs del maestro (incremental)	Archivado de binlogs cerrados, por instancia del maestro	**Cada 5 min**	7 días	Local cifrado + R2	`ops/backup-binlogs.sh` → recuperación a un punto en el tiempo con `ops/restore.sh … --hasta "AAAA-MM-DD HH:MM:SS"`
Bitácora (`audit_db`)	Completo	Cada hora	2 años	R2 con **Object Lock** (WORM)	`ops/backup.sh` · nadie puede borrar ni sobrescribir
Imágenes de productos	CDN redundante	Continuo	Indefinido	Cloudinary	Original en Cloudinary + URL en `catalogo_db`
Código fuente	Git	Cada push	Indefinido	GitHub	Incluye compose, Nginx y SQL de inicio
Imágenes Docker	Build de CI	Por versión	Últimas 5	GHCR	Reconstruibles desde el código
Secretos (`.env`)	Manual	Cada cambio	Historial	Gestor de contraseñas (fuera del servidor)	Sin `BACKUP_PASSPHRASE` los respaldos no se pueden restaurar

```
# /etc/cron.d/huellitas-respaldos
0 3 * * *      deploy cd /srv/huellitas && ops/backup.sh >> /var/log/huellitas-backup.log 2>&1
0 */4 * * *    deploy cd /srv/huellitas && ops/backup.sh >> /var/log/huellitas-backup.log 2>&1
*/5 * * * *    deploy cd /srv/huellitas && ops/backup-binlogs.sh >> /var/log/huellitas-binlogs.log 2>&1
0 4 1 * *      deploy cd /srv/huellitas && ops/simulacro-drp.sh --si >> /var/log/huellitas-drp.log 2>&1   # en el sitio B
```

Prueba de restauración: el simulacro mensual corre en el sitio B (nunca en producción) y deja su informe en `docs/simulacros/`.

---

4. Topología de recuperación



Paso de la conmutación al sitio B	Tiempo estimado
Decidir activar el sitio B (dueño + administrador)	10 min
`docker compose up -d` (imágenes ya descargadas)	3 min
`ops/restore.sh` del último respaldo de R2 (descarga + restauración)	5 min
Verificación (conteos, integridad de la bitácora, compra de prueba)	10 min
Cambio de DNS en Cloudflare y propagación (proxy, TTL 60 s)	2 min
Total	**≈ 30 min (< 60 min)**

---

5. Comunicación durante un incidente

1. **0–15 min:** el administrador confirma el incidente y avisa al dueño.
2. **15–30 min:** aviso en redes sociales y en la portada ("estamos en mantenimiento, tus pagos están seguros").
3. **Al cerrar:** informe con línea de tiempo, causa raíz, datos afectados (de la bitácora) y acciones para que no se repita.

Huellitas — Presupuesto de ciberseguridad (ALE / ROSI)

Supuestos del negocio

Dato	Valor	Origen
Ventas anuales en línea	USD 96,000	≈ USD 8,000/mes, tienda pequeña de productos para mascotas
Ticket promedio	USD 40	Catálogo de 24 productos con precios de USD 7 a 110
Pedidos al año	≈ 2,400	ventas / ticket
Venta por hora	≈ USD 11	96,000 / 8,760 h
Costo de una hora técnica	USD 25	Desarrollador local

Fórmulas

- **SLE** (pérdida por evento) = valor del activo × factor de exposición
- **ALE** (pérdida anual esperada) = SLE × ARO (ocurrencias por año)
- **Beneficio neto** = ALE antes − ALE después − costo anual del control
- **ROSI** = beneficio neto / costo del control. Si el control cuesta 0, el ROSI no está definido (∞) y se reporta el beneficio neto.

1. Análisis ALE por amenaza

#	Amenaza	SLE	ARO	ALE antes	Control implementado	Costo/año	ARO después	ALE después	Beneficio neto	ROSI
1	Fuga de datos de tarjetas	25,000	0.3	7,500	No se almacenan tarjetas: Stripe.js / PayPal (SAQ A)	0	0.01	250	7,250	∞
2	Cuenta de cliente comprometida	300	12	3,600	bcrypt 12, bloqueo 5 intentos/15 min, rate limit, detección multi-IP, JWT 1 h revocable	0	1.5	450	3,150	∞
3	Ransomware en el servidor	12,000	0.3	3,600	Respaldos cifrados en R2 + binlogs cada 5 min + DRP probado (RTO ≈ 14 s en el simulacro) → SLE baja a 300	12	0.3	90	3,498	29,150 %
4	DDoS (4 h sin ventas + horas técnicas)	250	4	1,000	Cloudflare gratuito + rate limit + límites por contenedor → SLE baja a 100	0	1	100	900	∞
5	Scraping de precios y catálogo	1,500	2	3,000	Caché Redis + rate limit por IP + stock exacto no publicado	0	0.4	600	2,400	∞
6	Webhook de pago falso	2,000	1	2,000	Firma `Stripe-Signature`, tolerancia 5 min, idempotencia, verificación de monto	0	0.01	20	1,980	∞
7	Precio manipulado en el carrito	1,000	2	2,000	Precio, cupón y envío calculados en el servidor	0	0	0	2,000	∞

#	Amenaza	SLE	ARO	ALE antes	Control implementado	Costo/año	ARO después	ALE después	Beneficio neto	ROSI
8	Credenciales de administrador comprometidas	8,000	0.2	1,600	Bitácora append-only, revocación de JWT, alertas `FRAUDE`, rol desde el token firmado	0	0.05	400	1,200	∞
	Total			**24,300**		**12**		**1,910**	**22,378**	

Montos en USD por año. Los controles de código (filas 1, 2, 5, 6, 7 y 8) no tienen costo de licencia; su costo es el tiempo de desarrollo del proyecto, que no se repite cada año.

Lectura: los controles reducen la pérdida anual esperada de **USD 24,300 a USD 1,910** (−92 %). La amenaza más cara antes de los controles era la fuga de tarjetas, y se elimina casi por completo con un control que no cuesta nada: no guardarlas.

2. Presupuesto anual

Categoría	Herramienta	Costo/año (USD)	Nota
Pagos seguros	Stripe / PayPal (sandbox; en producción, comisión por transacción)	0	La comisión es un costo de venta, no de seguridad
CDN + anti-DDoS + DNS	Cloudflare Free	0	Proxy, "Under Attack", reglas WAF básicas
Certificado TLS	Let's Encrypt	0	Renovación automática
Respaldos remotos	Cloudflare R2 (10 GB gratis + Object Lock)	12	Margen para crecer más allá del tramo gratuito
Imágenes / CDN	Cloudinary Free (25 créditos/mes)	0	
Caché y lista negra de JWT	Redis en Docker	0	Corre en el mismo servidor
Hosting HTTPS	VPS (parte proporcional) o Railway Hobby	120	Sitio A. El sitio B se enciende solo en una contingencia
Dominio	`.com`	15	
Escáner de vulnerabilidades	OWASP ZAP	0	`docs/zap/`
SAST	Semgrep CE en GitHub Actions	0	Corre en cada push (job `sast` del CI). Se eligió en lugar de CodeQL porque CodeQL no analiza PHP y en repos privados requiere GitHub Advanced Security
Correo transaccional	EmailJS Free (200/mes)	0	
Subtotal		**147**	
Contingencia (10 %)		15	
Total		**USD 162 / año**	Dentro del rango del enunciado (USD 23–265)

3. ROSI del programa completo

	USD/año
ALE sin controles	24,300
ALE con controles	1,910
Reducción de riesgo	22,390
Costo total de seguridad	162
Beneficio neto	**22,228**

	USD/año
ROSI = 22,228 / 162	**≈ 137× (13,721 %)**

Cada dólar del presupuesto de seguridad evita unos USD 137 de pérdidas esperadas al año.

4. Riesgo residual y siguiente inversión

Riesgo residual (ALE después)	Siguiente control	Costo estimado
Scraping (600)	Turnstile de Cloudflare en búsquedas masivas	0
Cuentas de cliente (450)	Aviso por correo ante un login desde un dispositivo nuevo	0 (EmailJS)
Admin comprometido (400)	✅ **Implementado:** MFA TOTP disponible para todas las cuentas, con aviso en el panel admin. Con MFA activo, el ARO estimado baja de 0.05 a 0.01 (ALE ≈ 80)	0

Huellitas — Controles ISO/IEC 27001:2022 (Anexo A)

Control	Implementación en Huellitas	Evidencia
A.5.15 Control de acceso	JWT con rol firmado (`cliente` / `admin`). El cliente solo ve sus pedidos, mascotas y citas (`WHERE id_cliente = sub`). Las rutas administrativas usan `Guard::usuario(['admin'])`. Entre servicios, JWT de servicio con lista de servicios permitidos	`services/shared/src/Guard.php`, `PedidosController::misPedidos()`, `CitasController::citaPropia()` · CP-07, CP-20, CP-29, CP-37, CP-40
A.5.29 Seguridad durante una disrupción	BCP/DRP con BIA (RTO/RPO por servicio), 6 escenarios y topología de sitio B	`docs/04-bcp-drp.md`
A.5.30 Preparación de las TIC para la continuidad	Simulacro de ransomware automatizado y cronometrado, con recuperación a un punto en el tiempo: RTO medido de unos 14 s, sin pérdida de datos (también de citas e historial clínico)	`ops/simulacro-drp.sh`, `docs/simulacros/`
A.5.34 Privacidad y protección de datos personales	Datos mínimos: sin tarjetas; la exportación de clientes solo lleva id, nombre, correo, estado y alta; el stock exacto no se publica; el JWT va en `sessionStorage`. Los datos de salud de las mascotas (diagnóstico y tratamiento) solo los ve el admin, cada lectura se audita (`HISTORIAL_CONSULTADO`), el carné del cliente no los incluye y la bitácora no los copia	`db/master/05-pagos_db.sql`, `AuthController::exportarClientes()`, `CitasController::carne()`, `historialAdmin()` · CP-28, CP-44, CP-45
A.8.3 Restricción del acceso a la información	Un usuario de BD por servicio, con permisos solo sobre su base. `audit_user` solo tiene INSERT/SELECT, y `pagos_user` y `citas_user` no tienen DELETE	`db/master/01-bases-y-usuarios.sh`, `db/audit/01-audit_db.sh` · CP-13, CP-25, CP-28
A.8.5 Autenticación segura	bcrypt de costo 12, bloqueo tras 5 intentos durante 15 min, mismo mensaje y mismo tiempo de respuesta exista o no la cuenta, JWT de 1 h revocable, verificación en dos pasos TOTP (RFC 6238) con aviso obligatorio en el panel para los administradores	`AuthController::login()`, `verificarMfa()`, `Totp.php` · CP-05, CP-08, CP-09, CP-34
A.8.9 Gestión de la configuración	Toda la infraestructura como código (compose, Nginx, Dockerfiles, SQL de inicio). Secretos en `.env` fuera del repositorio	`docker-compose.yml`, `docker-compose.prod.yml` · CP-30
A.8.13 Respaldo de la información	Respaldos completos AES-256 + SHA-256 y binlogs cada 5 min (recuperación a un punto en el tiempo), en local y remoto (R2), con retención de 30 días (2 años para la bitácora, con Object Lock). Restauración probada	`ops/backup.sh`, `ops/backup-binlogs.sh`, `ops/restore.sh`
A.8.15 Registro de actividades (logging)	`security-audit-service` con los 14 eventos obligatorios y 13 extra (10 del módulo de citas), append-only (triggers + permisos) y con cadena de hashes SHA-256 verificable	`services/audit/`, `/api/auditoria/integridad` · CP-12, CP-13, CP-40, CP-46
A.8.16 Actividades de monitoreo	Eventos `FRAUDE` (multi-IP, firma falsa, monto manipulado) y `ACCESO_DENEGADO`. Panel con indicadores e integridad de la bitácora	`/admin` (Resumen y Bitácora)
A.8.20 Seguridad de redes	Red Docker `interna` sin salida a Internet. Solo pagos y notificaciones están en `salida` (`citas-service` no tiene salida: los correos los pide a notificaciones). BD y Redis sin puertos publicados. Solo el gateway expone 80/443	`docker-compose.yml` · CP-14, CP-30
A.8.21 Seguridad de los servicios de red	Solo TLS 1.3, HSTS, WAF, rate limiting por servicio y CSP estricta	`gateway/` · CP-01, CP-02, CP-03, CP-10, CP-16
A.8.24 Uso de criptografía	bcrypt (contraseñas), TLS 1.3 (tránsito), AES-256 (respaldos), HMAC-SHA256 (JWT, webhooks de Stripe, comprobantes), SHA-256 (cadena de la bitácora)	`Jwt.php`, `Stripe.php`, `ops/backup.sh`
A.8.25 Ciclo de vida de desarrollo seguro	Modelado STRIDE antes de cada servicio, pruebas de seguridad automatizadas (263 comprobaciones en 46 casos) que corren en CI en cada push y escaneo DAST con ZAP	`docs/01…`, `tests/seguridad.sh`, `docs/zap/`

Control	Implementación en Huellitas	Evidencia
A.8.26 Requisitos de seguridad de las aplicaciones	OWASP Top 10 (7 controles) y PCI-DSS básico (6 requisitos) con referencias al código	`docs/02-owasp-top10.md`, `docs/03-pci-dss-pagos.md`
A.8.28 Codificación segura	100 % de prepared statements sin emulación, validación de tipo y rango, WAF en dos capas, errores genéricos y protección contra fórmulas en CSV	`Db.php`, `Validar.php`, `Router.php` · CP-10, CP-24

Simulacro DRP — ransomware en la BD maestra · 2026-10-17 11:26

Escenario 1 de docs/04-bcp-drp.md. Ejecutado con `ops/simulacro-drp.sh`.

Paso	Duración
Respaldo cifrado (AES-256)	0,5 s
Incidente → detección (API caída)	1,1 s
Maestro nuevo levantado	6,3 s
Restauración del respaldo + binlogs (PITR)	0,9 s
Réplica nueva sincronizada y API operativa	7,0 s
RTO medido (detección → operativo)	**14,2 s**

Objetivo de la BIA para la BD maestra: RTO 10 min · RPO 5 min.

Verificación

	usuarios · productos · stock · pedidos · líneas · pagos · mascotas · citas · consultas · vacunas
Antes del incidente	`116 24 2027 161 161 64 64 148 7 20`
Maestro restaurado	`116 24 2027 161 161 64 64 148 7 20`
Réplica reconstruida	`116 24 2027 161 161 64 64 148 7 20`

- Cambio posterior al respaldo completo (registro de `simulacro1792257965@correo.gt`): **recuperado desde los binlogs**.
- Bitácora antes: `true 2409` · después: `true 2410` (íntegra, registros). Al vivir en su propio volumen, el incidente no la alcanzó.
- Resultado: **RECUPERACIÓN COMPLETA, sin pérdida de datos**.
- Respaldo usado: `backups/20261017-112605` (verificado con SHA-256 antes de restaurar).

> RPO: el respaldo completo se complementa con los binlogs archivados (`ops/backup-binlogs.sh`, cron cada 5 min). Lo que se pierde en el peor caso es lo escrito desde el último archivado: **≤ 5 min**, como pide la BIA.

OWASP ZAP — escaneos del gateway (SEC-4)

Escaneo pasivo *baseline* (spider + reglas pasivas) contra el gateway HTTPS:

```
docker run --rm --add-host host.docker.internal:host-gateway -v "$PWD/docs/zap:/zap/wrk:rw" \
  ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t https://host.docker.internal:8443 \
  -j -m 3 -r zap-baseline-N.html -J zap-baseline-N.json -w zap-baseline-N.md -I
```

En producción el objetivo es la URL pública (`-t https://<dominio>`).

Corrida	Fecha	FAIL	WARN	PASS	Alto	Medio	Bajo	Info
1	2026-10-10	0	10	57	0	2	4	7
2	2026-10-10	0	7	60	0	**0**	3	6

Qué se corrigió entre la corrida 1 y la 2

Alerta (corrida 1)	Riesgo	Corrección
CSP: style-src unsafe-inline	Medio	`style-src 'self'` . `'unsafe-inline'` queda solo en `style-src-attr` (atributos `style` de React), que no permite inyectar hojas ni `<style>`
Sub Resource Integrity Attribute Missing	Medio	Las fuentes (Fredoka, Readex Pro) se alojan en el sitio (`@fontsource`). Ya no se carga la hoja externa de Google Fonts, que no admite SRI
Permissions Policy Header Not Set (`/assets/*`)	Bajo	Las cabeceras pasaron a `gateway/cabeceras_seguridad.inc` , incluido también en `location /assets/` (un `add_header` local anulaba los del server)
Cross-Origin-Resource-Policy Missing	Bajo	`Cross-Origin-Resource-Policy: same-origin`

Riesgo residual aceptado (corrida 2)

Alerta	Riesgo	Por qué se acepta
Cross-Origin-Embedder-Policy	Bajo	`require-corp` o `credentialless` bloquean los iframes de Stripe Elements, de los botones de PayPal y de Google Maps, que no envían las cabeceras necesarias. Sin COEP solo se pierde el aislamiento entre orígenes (`SharedArrayBuffer`), que la tienda no usa
Cross-Origin-Opener-Policy	Bajo	Se usa `same-origin-allow-popups` en lugar de `same-origin` porque PayPal abre una ventana emergente para el login. Igual protege contra que otra página controle la ventana de la tienda
Timestamp Disclosure - Unix	Bajo	Falso positivo: números de 10 dígitos dentro del bundle minificado de React, no fechas del servidor
Information in Browser localStorage	Info	Carrito y favoritos (IDs de producto y cantidades). El JWT **no** se guarda ahí, sino en `sessionStorage` , que se borra al cerrar la pestaña
Suspicious Comments	Info	Textos dentro de librerías minificadas (React Router), sin datos internos
Cache-control / Storable / Retrieved from Cache / Modern Web App	Info	`/assets/*` tienen hash en el nombre y se cachean un año a propósito. La API responde `Cache-Control: no-store`